Ask the Expert

Data Manipulation Techniques

Carleigh Jo Crabtree, Technical Training Consultant

Copyright © SAS Institute Inc. All rights reserved.

Carleigh Jo Crabtree

Technical Training Consultant

Carleigh Jo works in content development as a technical training consultant, creating and delivering technical courses designed to make complex data concepts clear and actionable. Her expertise spans DATA step processing, program efficiency techniques, optimizing programs with SAS macros and programming strategies that can apply to a range of real-world scenarios. She regularly presents at conferences and user groups, sharing practical insights that help practitioners solve everyday data challenges. Carleigh Jo is currently pursuing graduate studies in Engineering Management at NC State.

Data Manipulation Techniques

Ask the Expert

Carleigh Jo Crabtree
SAS Technical Training Consultant
CarleighJo.Crabtree@sas.com

Copyright © SAS Institute Inc. All rights reserved.

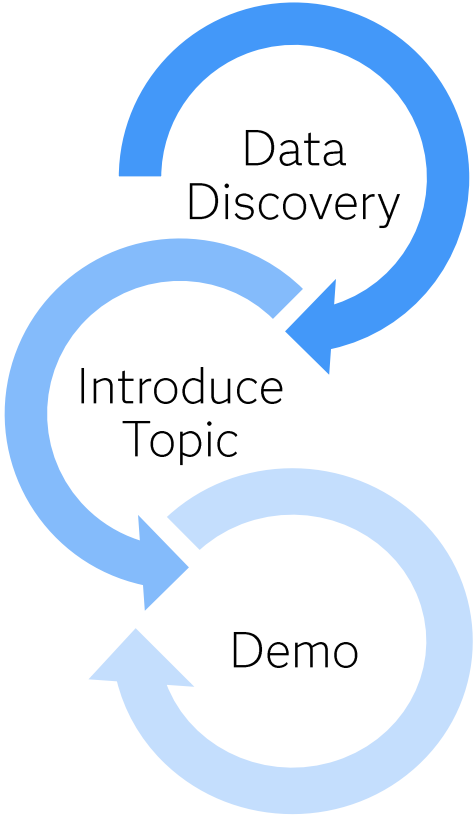
About Carleigh Jo

SAS Technical Training Consultant

Pursuing Master of Engineering Management, Analytics

Travel, baking

Workshop Flow



Data Discovery

Topics

- Controlling Data Step Processing
- Summarizing Data
- Manipulating Data with Functions
- Creating Custom Formats
- Combining Tables
- Processing Repetitive Code

Controlling Data Step Processing

Default DATA Step Behavior:

1. Read one row from input table
2. Manipulate values
3. Output row to output table
4. Return to the top of the DATA step

```
DATA output;
    SET input;
    …other statements…
RUN;
```

Implicit OUTPUT;
Implicit RETURN;

Controlling Data Step Processing

How can we create and populate **multiple** output tables?

Explicit output

Controlling Data Step Processing

DATA table1 <table2...>;

OUTPUT table1 <table2...>;

```
DATA trucks other;
    SET sashelp.cars;
    IF Type="Truck"then output trucks;
    ELSE output other;
RUN;
```

Explicit output

Demo_01

Populating Multiple Tables with Explicit Output

Topics

- Controlling Data Step Processing
- Summarizing Data
- Manipulating Data with Functions
- Creating Custom Formats
- Combining Tables
- Processing Repetitive Code

Summarizing Data

column+expression;

```
DATA output;
     SET input;
     DayNum+1;
RUN;
```

DayNum
1
2
3
4

Sum Statement:

1. Creates **DayNum**
2. Sets initial value to **0**
3. Retains **DayNum**
4. Adds **1** to **DayNum** for each row
5. Ignores missing values

Summarizing Data

How can we create an accumulating column within **groups**?

BY statement

Summarizing Data

1 Sort Data

Use PROC SORT with the BY statement

2 BY Statement in DATA Step

Use BY statement in the DATA step to group data

3 First./Last. BY Variables

Use First./Last. variables to determine what happens at the beginning or end of a group

Summarizing Data

1 Sort Data

Use PROC SORT with the BY statement

```
PROC SORT DATA= countries
          OUT= sortedCountries;
    BY Country;
RUN;
```

Sorts the table into groups of country

2 BY Statement in DATA Step

Use BY statement in the DATA step to group data

```
DATA groupedCountries;
    SET sortedCountries;
    BY Country;
RUN;
```

Processes the data in the sorted table by groups

3 First./Last. BY Variables

Use First./Last. variables to determine what happens at the beginning or end of a group

```
DATA groupedCountries;
    SET sortedCountries;
    BY Country;
RUN;
```

First./Last. variables can be used to identify when each BY group begins and ends.

Country	First.Country	Last.Country
Australia	1	0
Australia	0	0
Australia	0	0
Australia	0	1
Brazil	1	0
Brazil	0	0
Brazil	0	1

Summarizing Data

What statement can be used to process First./Last. BY variables?

Subsetting
IF
statement

```
if First.Country=1 then …
```

Demo_02

Accumulating Column within Groups

Topics

- Controlling Data Step Processing
- Summarizing Data
- Manipulating Data with Functions
- Creating Custom Formats
- Combining Tables
- Processing Repetitive Code

Manipulating Data with Functions

```
function(argument1, argument2, ...);
```

Manipulating Data with Functions

Numeric

LARGEST()
Returns the K largest value of a group

MEAN()
Returns the average across columns

ROUND()
Rounds to the nearest *nth*

Character

SCAN()
Extracts specified word from a string

FIND()
Returns the starting position of a substring in a string

SUBSTR()
Extracts characters from a string

Date

DATEPART()
Extract date from datetime value

INTNX()
Shift date to next interval

Manipulating Data with Functions

How can we specify a range of columns?

Column Lists

Manipulating Data with Functions

Syntax	Example
x1-x*n*	mean(of temp1- temp9)
x--a	mean(of temp--dailytemp)
PREFIX:	mean(of temp:)
ALL	format _ALL_ comma16.;
NUMERIC	format _NUMERIC_ dollar20.;
CHARACTER	propcase(_CHARACTER_)

Column Lists

Demo_03

Manipulating Data with Numeric Functions

Demo_04

Manipulating Data with Character Functions

Demo_05

Manipulating Data with Date Functions

Topics

- Controlling Data Step Processing
- Summarizing Data
- Manipulating Data with Functions
- Creating Custom Formats
- Combining Tables
- Processing Repetitive Code

Creating Custom Formats

How can we create a user defined format?

FORMAT procedure

Creating Custom Formats

```
PROC FORMAT;
    VALUE formatName value-or-range= 'formatted value'
                     value-or-range= 'formatted value'
                     . . . ;
RUN;
```

- The name can be up to 32 characters.
- Character formats begin with a $ followed by a letter or underscore.
- Numeric formats begin with a letter or underscore.
- The name cannot end in a number or match an existing SAS format.

Creating Custom Formats

```
PROC FORMAT;
    VALUE formatName value-or-range= 'formatted value'
                     value-or-range= 'formatted value'
                      . . . ;
RUN;
```

Individual value or range of values you want to format

Creating Custom Formats

```
PROC FORMAT;
    VALUE formatName value-or-range= 'formatted value'
                     value-or-range= 'formatted value'
                     . . . ;
RUN;
```

Format to apply to the individual value or range of values

Creating Custom Formats

```
PROC FORMAT;
    VALUE hrange 50-<58 = 'Below Average'
                 58-60  = 'Average'
                 60<-70 = 'Above Average';
RUN;
```

Put a **<** next to the value to exclude when defining a range

Creating Custom Formats

To **use** a format, add a **period** after the format name

```
FORMAT colName fmtName.;
```

```
PUT(colName, fmtName.);
```

Demo_06

Creating Custom Formats- Single Values

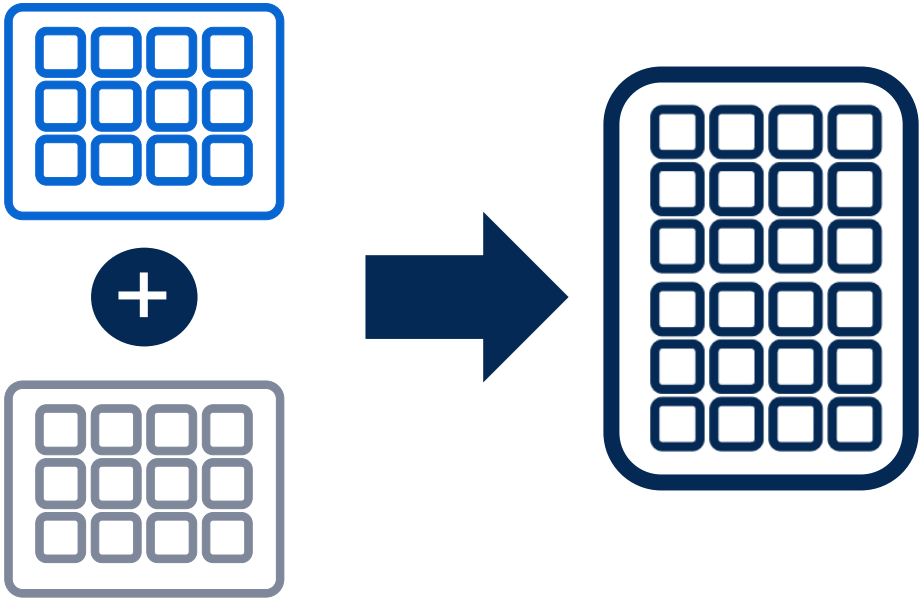
Demo_07

Creating Custom Formats- Range of Values

Topics

- Controlling Data Step Processing
- Summarizing Data
- Manipulating Data with Functions
- Creating Custom Formats
- Combining Tables
- Processing Repetitive Code

Combining Tables



Concatenating Tables

Combining Tables

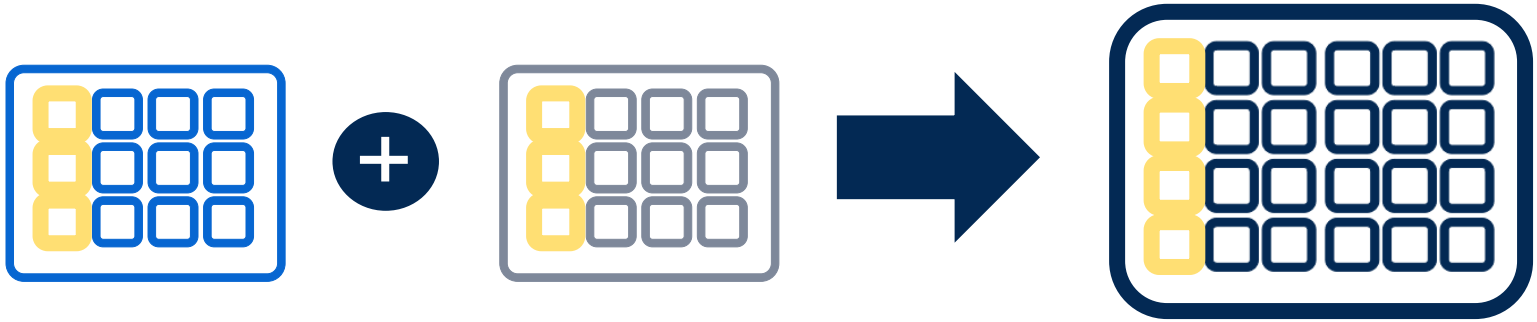
```
DATA output;
    SET input1 input2 …;
RUN;
```

Any number of input tables

Demo_08

Concatenating Tables

Combining Tables



Merging Tables

Combining Tables

```
DATA output;
    MERGE input1 input2 …;
    BY commonColumn(s);
RUN;
```

List common column(s)

Any number of input tables with one or more common columns

Combining Tables

If there is a **BY** statement in the DATA step what must be true?

Input tables must be sorted by the column(s) on the BY statement

Combining Tables

one-to-one

A	B	C
		1
		2
		3

C	D	E
1		
2		
3		

nonmatching rows

A	B	C
		1
		2
		4

C	D	E
2		
3		
4		

one-to-many

A	B	C
		1
		2

C	D	E
1		
1		
2		

Demo_09

One-to-Many Merge

Bonus

- Controlling Data Step Processing
- Summarizing Data
- Manipulating Data with Functions
- Creating Custom Formats
- Combining Tables
- Processing Repetitive Code

Processing Repetitive Code

How can we eliminate repetitive code?

DO Loops

Processing Repetitive Code

Types of DO Loops

Iterative DO Loop

```
DO indexCol = start TO stop BY increment;
    … repetitive code …
END;
```

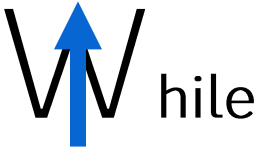
Processing Repetitive Code

Types of DO Loops

Conditional DO Loop

```
DO UNTIL(expression);
    … repetitive code …
END;
```

```
DO WHILE(expression);
    … repetitive code …
END;
```



Processing Repetitive Code

Types of DO Loops

Iterative and **Conditional** DO Loop

```
DO i = 1 TO 6 UNTIL(x > 100);
    … repetitive code …
END;
```

```
DO i = 1 TO 6 WHILE(x < 100);
    … repetitive code …
END;
```

Demo_10

Iterative DO Loop

Demo_11

Conditional DO Loops

Demo_12

Iterative and Conditional DO Loops

Thank you!

Materials GitHub

Let’s connect!

Copyright © SAS Institute Inc. All rights reserved.